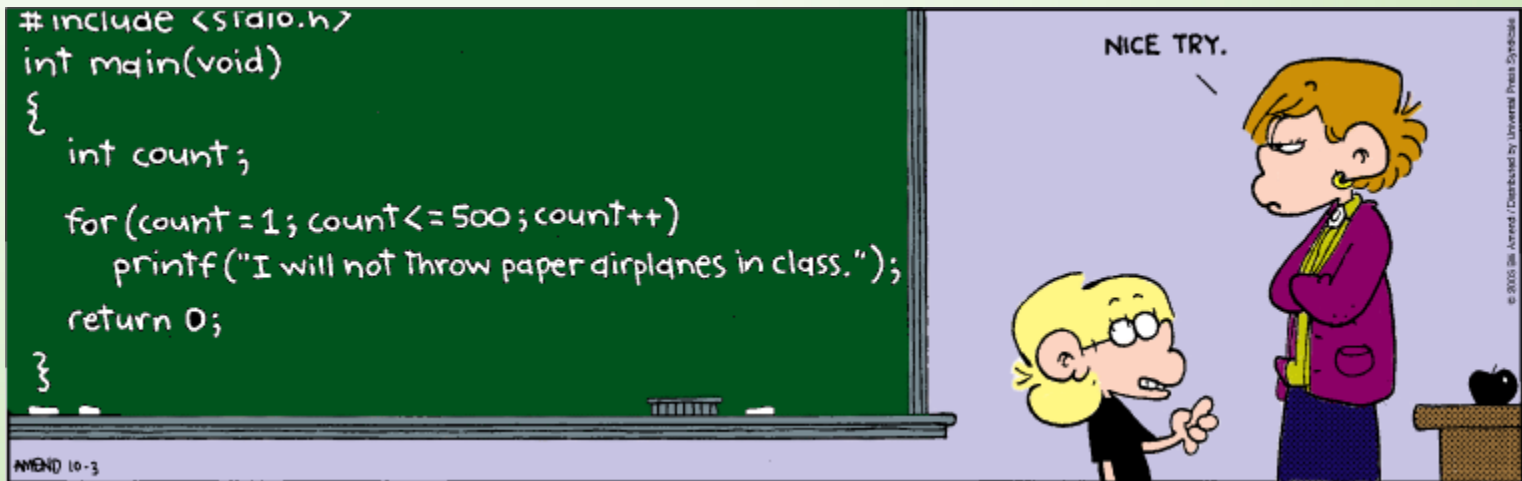


Chapter 7:

Arrays

(Single Dimensional)





Objectives

- ✦ To describe why arrays are necessary in programming (§7.1).
- ✦ To declare array reference variables and create arrays (§§7.2.1–7.2.2).
- ✦ To obtain array size using **arrayRefVar.length** and know default values in an array (§7.2.3).
- ✦ To access array elements using indexes (§7.2.4).
- ✦ To declare, create, and initialize an array using an array initializer (§7.2.5).
- ✦ To program common array operations (displaying arrays, summing all elements, finding the minimum and maximum elements, random shuffling, and shifting elements) (§7.2.6).
- ✦ To simplify programming using the foreach loops (§7.2.7).
- ✦ To apply arrays in application development (**AnalyzeNumbers**, **DeckOfCards**) (§§7.3–7.4).
- ✦ To copy contents from one array to another (§7.5).
- ✦ To develop and invoke methods with array arguments and return values (§§7.6–7.8).
- ✦ To define a method with a variable-length argument list (§7.9).
- ✦ To search elements using the linear (§7.10.1) or binary (§7.10.2) search algorithm.
- ✦ To sort an array using the selection sort approach (§7.11).
- ✦ To use the methods in the **java.util.Arrays** class (§7.12).
- ✦ To pass arguments to the main method from the command line (§7.13).



Program 1: Array Introduction

- ☞ Read one hundred numbers from the user, compute the average of the numbers, and then find out how many numbers are above the average.
- ☞ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 1: Array Introduction

☞ **Step 1: Problem-solving Phase (Algorithm)**

- We need to read 100 values from the user
- That is easy. We use a **for** loop (iterate 100 times).
- We keep a variable **sum** to sum all values entered by user.
- So for each iteration of the loop, we read the value, and we add this value to the variable sum
sum += userValue;
- Then, after the **for** loop, we simply divide **sum** by 100 to get the average.



Program 1: Array Introduction

☞ Step 1: Problem-solving Phase (Algorithm)

- Well, now that we have the average, how can we find and print the user-values that were above the average?
 - ◆ Hmmm. We did not save each of those 100 values.
 - ◆ We just read them and added the values to sum.
 - ◆ Okay. Well, one possible solution is to make 100 **int** variables!

number0, number1, ..., number99

- Is that an acceptable solution?
- What if the number of user-inputted values was 10,000?
- Would you make 10,000 **int** variables?

This is a problem. But it is EASY to solve with Arrays!



Declaring Array

- ➡ An array is used to store a collection of data
 - The collection of data can be any type
 - However, we usually consider an array as a collection of variables of the same data type
- ➡ To use an array in a program, you must declare a **variable to reference the array**, and you must specify the array's element type
 - Syntax:

```
elementType[] arrayRefVar;
```



Declaring Array (cont.)

- Example:

```
double[] myList;
```

- all elements of the array will have the same data type

- Note:

- ◆ The brackets, **[]**, should appear after the data type
 - This is preferred in Java
- ◆ Also allowed to place the brackets after the variable name

```
double myList[];
```

 - This style comes from C/C++
- ◆ Both are correct. The first method is preferred with Java.



Creating Arrays

- So now you declared an array, Syntax:

elementType[] arrayRefVar;

- and you want to begin saving values into the array,
But first, you must create the array

- You create an array by using the **new** operator

- Syntax:

arrayRefVar = new elementType[arraySize];

- ◆ This statement does two things:

1. It created an array using **new elementType[arraySize];**
2. It assigns (saves) the reference of the newly created array to the variable **arrayRefVar**



Creating Arrays (cont.)

- You can also declare an array variable and create the array all in once step as follows:

```
elementType[] arrayRefVar = new elementType[arraySize];
```

- Example:

```
◆ double[] myList = new double[10];
```

- This statement declares an array variable, **myList**,
- and creates an array of ten elements of **double** type,
- and then saves the reference of the array to **myList**



Accessing Array Elements

- Array elements are accessed through their index
- Array indices are 0-based:
 - ◆ this means that start from **0** and go up to **arraySize – 1**
- Example:
 - ◆ If an array has 10 elements:
 - the first element is saved at index 0
 - the last element is saved at index 9.
 - ◆ **myList** holds ten **double** values, and these values are saved in the array from indices **0** to **9**.



Save Values into the Array

- How do we save values into the array???
- Easy, here is the syntax:

arrayRefVar[index] = value;

- Consider the following code on the right
- Here we have assigned specific values to each array index

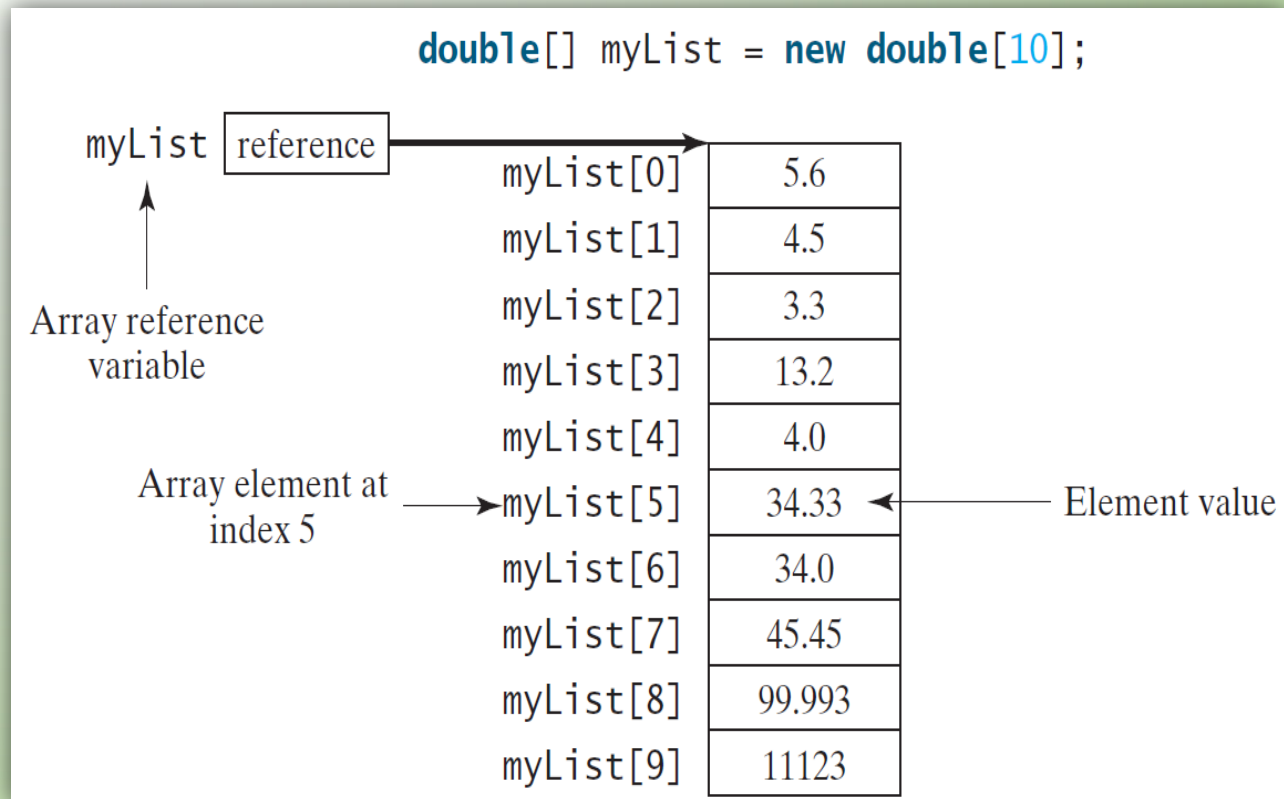
```
myList[0] = 5.6;  
myList[1] = 4.5;  
myList[2] = 3.3;  
myList[3] = 13.2;  
myList[4] = 4.0;  
myList[5] = 34.33;  
myList[6] = 34.0;  
myList[7] = 45.45;  
myList[8] = 99.993;  
myList[9] = 11123;
```



Save Values into the Array (cont.)

- The array **myList** has ten elements of **double** type
 - ◆ Those elements are saved from *index 0* to *index 9*

```
myList[0] = 5.6;  
myList[1] = 4.5;  
myList[2] = 3.3;  
myList[3] = 13.2;  
myList[4] = 4.0;  
myList[5] = 34.33;  
myList[6] = 34.0;  
myList[7] = 45.45;  
myList[8] = 99.993;  
myList[9] = 11123;
```





Processing Arrays

- Array processing is almost always done with a **for** loop, and this is for two reasons:
 1. All of the elements in the array are the same data type
 - So it is easy to use a **for** loop to repeatedly access them
 2. The size of the array is known. Therefore, it is natural to use a **for** loop.



Program 1: Array Introduction (cont.)

☞ Step 1: Problem-solving Phase

– Algorithm:

- ◆ Make an array of size 100
 - Don't worry about how to do this in detail for now
 - We have two weeks to explain the details!
- ◆ Save all user-inputted values into the array
- ◆ Compute the average
- ◆ Loop back through all values in the array and identify and print all values above the average



Program 1: Array Introduction (cont.)

👉 Step 2: Implementation

```
import java.util.*;
public class ArrayIntroduction {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int[] myList = new int[100];
        int sum = 0, userValue;
        double average;

        for (int i = 0; i < 100; i++) {
            userValue = in.nextInt();
            myList[i] = userValue;
            sum += myList[i];
        }

        average = sum / 100;

        for (int i = 0; i < 100; i++) {
            if (myList[i] > average)
                System.out.println (myList[i] + " is greater than the average.")
        }
    }
}
```

Program 1: Array Introduction (cont.)

👉 Run the Program:

```
Enter the number of items: 10 ↵ Enter  
Enter the numbers: 3.4 5 6 1 6.5 7.8 3.5 8.5 6.3 9.5 ↵ Enter  
Average is 5.75  
Number of elements above the average is 6
```



[Click here](#) to view and trace code



Array Basics

☞ Note:

- When you declare a variable of a primitive data type, space is reserved in memory for that variable
- This does **not** happen when you declare an array
- An array declaration only creates a storage location for the **reference** to the array.
 - ◆ The array declaration does not "create an array", Array variables only hold a reference to actual array
 - ◆ And if the array has not been created (**new**), this reference to the array will store the value **null**
 - This means the reference variable does not point/refer anywhere
 - Because, in fact, there is no array...yet



Array Default Values

- When an array is created, the elements of the array are assigned default values
- These values depend on the data type of the elements
 - ◆ For numeric types:
 - **0** is assigned
 - ◆ For **char** types:
 - **\u0000** is assigned
 - ◆ For **boolean** types:
 - **false** is assigned



Array Size

- When an array is created, the array size must be given

```
int[] values = new int[100];
```

- ◆ Here, the “100” refers to the number of elements in the array

- Note: the size of an array **cannot** be changed after the array is created

- ◆ The size is "fixed" and cannot be changed

- You can get the size of your array with the following:

```
arrayRefVar.length
```

- Example

Values.length is 100 (because there are 100 elements)



Array Initializers

- Java allows you to declare, create, and initialize the array in one step

- **Syntax:**

elementType[] arrayRefVar = {value0, value1, ..., valuek};

- **Example:**

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

- ◆ This declares, creates, and initializes the array **myList** with four double elements

- This is equivalent to:

```
double[] myList = new double[4];
```

```
myList[0] = 1.9;
```

```
myList[1] = 2.9;
```

```
myList[2] = 3.4;
```

```
myList[3] = 3.5;
```



Array Initializers (cont.)

– Caution/Warning:

- ◆ The **new** operator is not used in the array-initializer syntax
- ◆ And you must declare, create, and initialize all in one step
elementType[] arrayRefVar = {value0, value1, ..., valuek};
- ◆ Splitting the steps would cause a syntax error
- ◆ **Example:** the following statement is wrong:

```
double[] myList;  
myList = {1.9, 2.9, 3.4, 3.5};
```



Trace Program with Arrays

- Consider the following example program
 - We create an array of size 5
 - We then use a loop to iterate over all values of the array and modify the values

Declare array variable values, create an array, and assign its reference to values

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the array is created



0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

i becomes 1

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

i (=1) is less than 5

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

After this line is executed, value[1] is 1

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

After i++, i becomes 2

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

i (= 2) is less than 5

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

After this line is executed,
values[2] is 3 (2 + 1)

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

After this, i becomes 3.

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

i (=3) is still less than 5.

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

After this line, values[3] becomes 6 (3 + 3)

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

After this, i becomes 4

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

i (=4) is still less than 5

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

After this, values[4] becomes 10 (4 + 6)

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

After i++, i becomes 5

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

$i (=5) < 5$ is false. Exit the loop

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

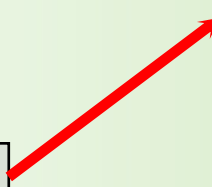
After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

After this line, values[0] is 11 (1 + 10)

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```



0	11
1	1
2	3
3	6
4	10



Processing Arrays

1. Printing arrays
2. Initializing arrays with input values
3. Initializing arrays with random values
4. Summing all elements
5. Finding the largest element
6. Finding the smallest index of the largest element
7. Random shuffling
8. Shifting elements



Processing Arrays

Printing arrays

- To print an array, you simply use a loop to iterate over each cell of the array as follows:

```
for (int i = 0; i < myList.length; i++) {  
    System.out.println(myList[i] + " ");  
}
```

- Notice the loop-continuation-condition:
 - ◆ We use **myList.length**
 - ◆ Again, this is not a method
 - ◆ Rather, it is built-in functionality for arrays



Processing Arrays

Initializing arrays with input values

- Here, we ask the user to enter the values, which will be saved in the array:

```
input = new Scanner(System.in);  
double[] myList = new double[10];  
System.out.print("Enter " + myList.length + " values: ");  
for (int i = 0; i < myList.length; i++)  
    myList[i] = input.nextDouble();
```

- We simply iterate **myList.length** times
- Each time, we scan a new value from the user



Processing Arrays

Initializing arrays with input values

- Here, we create an array, and we initialize the array with random values

```
double[] myList = new double[10];  
  
for (int i = 0; i < myList.length; i++) {  
    myList[i] = Math.random() * 100;  
}
```

- Again, we simply iterate **myList.length** times
- Each time, we save a new random value into the **ith** index of the array



Processing Arrays

Summing all elements of an array

- Use a variable named total to store the sum
- We add each element to total as follows:

```
double total = 0;

for (int i = 0; i < myList.length; i++) {
    total += myList[i];
}
```



Processing Arrays

Finding the largest element of an array

- Use a variable, **max**, to store the largest element
 - ◆ Initially, **max** is **myList[0]**
- Use a for loop to iterate over the array
 - ◆ Compare each array element to **max**
 - ◆ If it is bigger than **max**, then we make it the new max!

```
double max = myList[0];  
  
for (int i = 1; i < myList.length; i++) {  
    if (myList[i] > max)  
        max = myList[i];  
}
```



Processing Arrays

Finding the smallest index of the largest element

- This is similar to the last example, but now we assume possible repeated values

- Example:

`myList = {1, 5, 3, 4, 5, 5}`

- We want to know the largest element AND the index of the first location of this largest element
- Element 5 is at index 1, index 4, and index 5
- We want the first location: index 1



Processing Arrays

Finding the smallest index of the largest element

- The code is very similar to last example
- Only difference is the saving of **indexOfMax**

```
double max = myList[0];
int indexOfMax = 0;

for (int i = 1; i < myList.length; i++) {
    if (myList[i] > max) {
        max = myList[i];
        indexOfMax = i;
    }
}
```



Processing Arrays

Random Shuffling

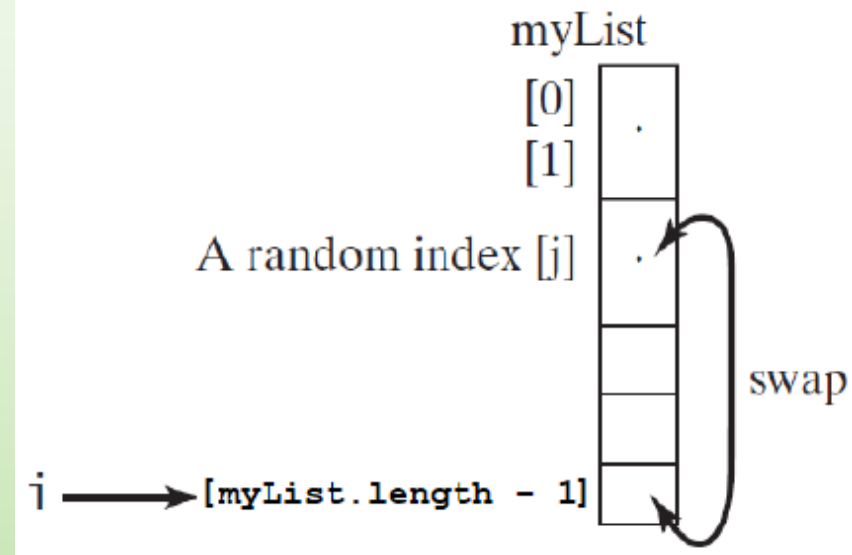
- For many applications, you will need to randomly reorder the elements in an array.
- This is called shuffling
- Algorithm:
 - ◆ For each element, `myList[i]`
 - ◆ Randomly generate an index `j`
 - Of course, the randomly generated number must be between `0` and `myList.length - 1`
 - ◆ Finally, swap the values `myList[i]` and `myList[j]`
 - ◆ Repeat this `myList.length` times (or more times!)

Processing Arrays

Random Shuffling

– Implementation:

```
for (int i = myList.length - 1; i > 0; i--)  
{ // Generate an index j randomly with 0 <= j <= i  
    int j = (int) (Math.random() * (i + 1));  
    // Swap myList[i] with myList[j]  
    double temp = myList[i];  
    myList[i] = myList[j];  
    myList[j] = temp;  
}
```





Processing Arrays

Shifting Elements

- Sometimes you need to shift the elements left or right inside an array
- The following example shifts all elements one position over to the left
 - ◆ The first element (saved at temp) is then copied to last index

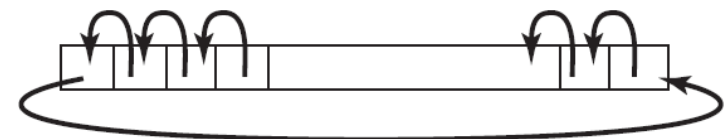
```
double temp = myList[0]; // Retain the first element
```

```
// Shift elements left
```

```
for (int i = 1; i < myList.length; i++) {  
    myList[i - 1] = myList[i];  
}
```

```
// Move the first element to fill in the last position  
myList[myList.length - 1] = temp;
```

myList





Example: Month name by its number

- Using arrays can also simplify coding for certain tasks you may not immediately think of
- Consider you wish to obtain a given month name by its number in the year
 - ◆ Normally, you would use a long if/else-if statement

```
if (monthNumber == 1)
    System.out.println("January");
else if (monthNumber == 2)
    System.out.println("February");
...
else
    System.out.println("December");
```



Example: Month name by its number

- Now, consider if you save all month names inside an array of **String** values
- Then, the month name can be accessed via the index
- The following code asks a user to enter a month number and then displays the name of the month:

```
String[] months = {"January", "February", ..., "December"};  
System.out.print("Enter a month number (1 to 12): ");  
int monthNumber = input.nextInt();  
System.out.println("The month is " + months[monthNumber - 1]);
```

- Question: why did we use index "monthNumber - 1" ?



Enhanced **for** Loop

- ➡ Java supports a convenient **for** loop
 - known as a **for each loop**
- ➡ This loop enables you to iterate through the loop without using an array index variable
- ➡ Example: The following code prints all values in the array **myList**:

```
for (double e: myList) {  
    System.out.println(e);  
}
```

- You can read the code as "for each element **e** in **myList**, do the following..."
- Note that the variable, **e**, must be declared as the same type as the elements in **myList**



CAUTION: Common error

➡ Most common error is to attempt to access/print elements of an array that do not exist

➡ Example:

– You have an array of size 10 called **myList**, and you try to print the *last* element as follows:

```
System.out.println(myList[10]);
```

– That would be an error, as there is no index 10 in this array. The biggest index is index 9.

– This is called an ***ArrayIndexOutOfBoundsException***

– Be careful!



CAUTION: Common error

- Programmers often reference the first element in an array with the index 1
 - ◆ The first element is at index 0
- This is called the *off-by-one-error*
- ☞ Another common off-by-one error is to use **<=** where **<** should be used
 - Example:

```
for (int i = 0; i <= list.length; i++)  
    System.out.println(list[i] + " ");
```



Check Point

➡ What is the output of the following code?

```
int x = 30;  
int[] numbers = new int[x];  
x = 60;  
System.out.println("x is " + x);  
System.out.println("The size of numbers is " + numbers.length);
```

Show Answer

x is 60

The size of numbers is 30



Check Point

- ➡ Indicate **true** or **false** for the following statements:
- Every element in an array has the same type.
 - The array size is fixed after an array reference variable is declared.
 - The array size is fixed after it is created.
 - The elements in an array must be a primitive data type.

Show Answer

- (a) Answer: True
- (b) Answer: False
- (c) Answer: True
- (d) Answer: False



Check Point

☞ Identify and fix errors in the following code:

```
1  public class Test {  
2      public static void main(String[] args) {  
3          double[100] r;  
4  
5          for (int i = 0; i < r.length(); i++);  
6              r(i) = Math.random * 100;  
7      }  
8  }
```

Show Answer

Line 3: the array declaration is wrong. It should be `double[]`. The array needs to be created before its been used. e.g. `new double[10]`

Line 5: The semicolon (;) at the end of the for loop heading should be removed.

Line 5: `r.length()` should be `r.length`.

Line 6: `random` should be `random()`

Line 6: `r(i)` should be `r[i]`.



Check Point

➡ What is the output of the following code:

```
1  public class Test {  
2      public static void main(String[] args) {  
3          int list[] = {1, 2, 3, 4, 5, 6};  
4          for (int i = 1; i < list.length; i++)  
5              list[i] = list[i - 1];  
6  
7          for (int i = 0; i < list.length; i++)  
8              System.out.print(list[i] + " ");  
9      }  
10 }
```

Show Answer

1 1 1 1 1 1



Program 2: Deck of Cards

- ➡ Given a deck of playing cards, write a program that will randomly select four cards from the deck.
 - A deck has 52 cards
 - 13 cards from each suit
 - ◆ Suits are: Spades, Hearts, Diamonds, and Clubs
- ➡ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



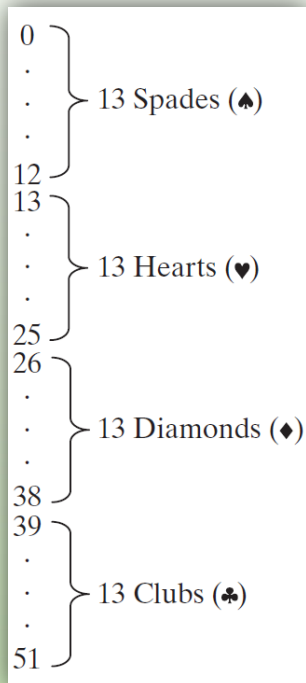
Program 2: Deck of Cards

☞ Step 1: Problem-solving Phase

– Details:

◆ We can represent the 52 cards with an array of **int** values

- The first 13 cards (index 0 to 12) will represent the Spades
- The next 13 cards (index 13 to 25) will represent the Hearts
- The next 13 cards (index 26 to 38) will represent the Diamonds
- The next 13 cards (index 39 to 51) will represent the Clubs



```
int[] deck = new int[52];  
  
// Initialize cards  
for (int i = 0; i < deck.length; i++)  
    deck[i] = i;
```



Program 2: Deck of Cards

☞ Step 1: Problem-solving Phase

– Details:

- ◆ So how can we determine that card 38 is the King of Diamonds? Or that card 15 is the 3 of Hearts?

- ◆ Make two new arrays: **suits** and **ranks**

```
String[] suits = {"Spades", "Hearts", "Diamonds", "Clubs"};  
String[] ranks = {"Ace", "2", "3", "4", "5", "6", "7", "8",  
                  "9", "10", "Jack", "Queen", "King"};
```

- ◆ Given a card number (38 for example), we can find:

- Suit of 38 $\Rightarrow 38 / 13 = 2$

- this is the 2nd index of the suits array, which is "Diamonds"

- Rank of 38 $\Rightarrow 38 \% 13 = 12$

- this is the 12th index of the ranks array, which is for "King"



Program 2: Deck of Cards

☞ Step 1: Problem-solving Phase

– Details:

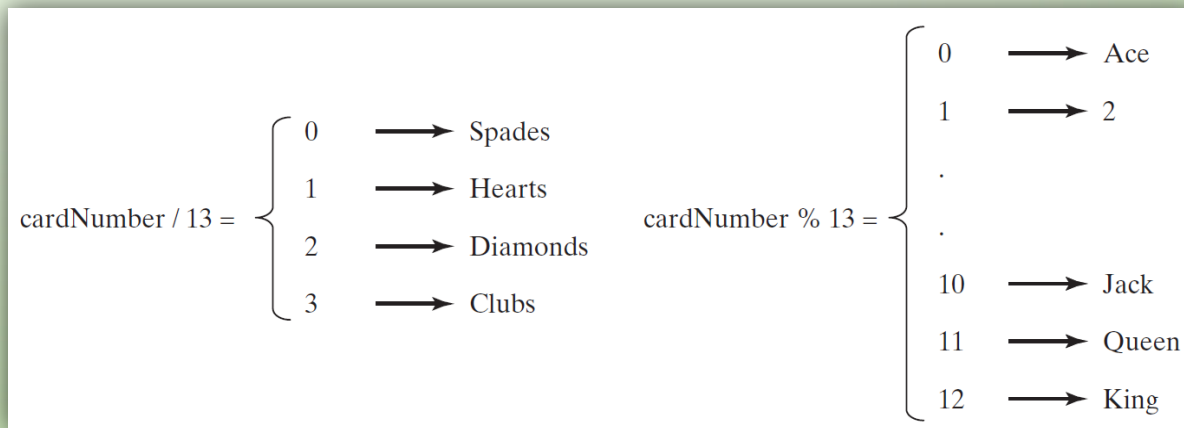
◆ Given a card number (15 for example), we can find:

– Suit of 15 $\Rightarrow 15 / 13 = 1$

- this is the 1st index of the **suits** array, which is "Hearts"

– Rank of 15 $\Rightarrow 15 \% 13 = 2$

- this is the 2nd index of the **ranks** array, which is for "3"





Program 2: Deck of Cards

☞ Step 1: Problem-solving Phase

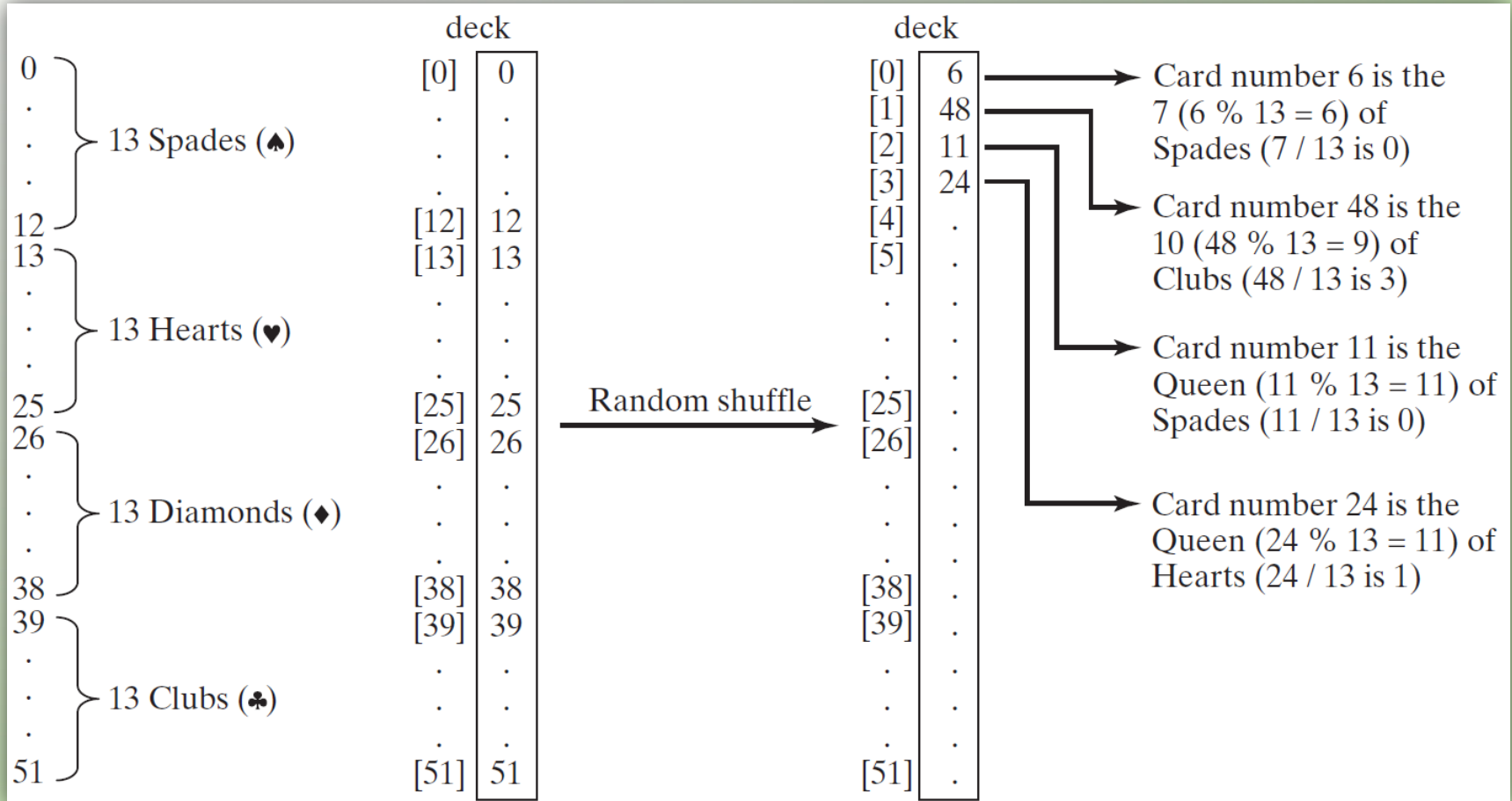
– Details:

- ◆ So if we are given an index, we are able to identify the rank and the suite of the card!
- ◆ What is left?
 - The program requested to **randomly** select four cards
- ◆ You could make a for loop
 - Iterate 4 times, and each time, generate a random index
 - Then, for that index, determine the rank and the suit
- ◆ Also, you could first shuffle all elements in the array!
- ◆ So now the array is randomly shuffled
 - Then just select the first four cards!



Program 2: Deck of Cards

➡ Step 1: Problem-solving Phase





Program 2: Deck of Cards

👉 Step 2: Implementation

```
public class DeckOfCards {
    public static void main(String[] args) {
        int[] deck = new int[52];
        String[] suits = {"Spades", "Hearts", "Diamonds", "Clubs"};
        String[] ranks = {"Ace", "2", "3", "4", "5", "6", "7", "8", "9", "10", "Jack", "Queen", "King"};

        // Initialize cards
        for (int i = 0; i < deck.length; i++)
            deck[i] = i;

        // Shuffle the cards
        for (int i = 0; i < deck.length; i++) {
            // Generate an index randomly
            int index = (int)(Math.random() * deck.length);
            int temp = deck[i];
            deck[i] = deck[index];
            deck[index] = temp;
        }

        // Display the first four cards
        for (int i = 0; i < 4; i++) {
            String suit = suits[deck[i] / 13];
            String rank = ranks[deck[i] % 13];
            System.out.println("Card number " + deck[i] + ": " + rank + " of " + suit);
        }
    }
}
```



Program 2: Deck of Cards

☞ Run the Program:

```
Card number 6: 7 of Spades  
Card number 48: 10 of Clubs  
Card number 11: Queen of Spades  
Card number 24: Queen of Hearts
```



[Click here](#) to view and trace code



Program 2: Deck of Cards

– Comments:

- ◆ We do not need the **suits** and **ranks** arrays
- ◆ If we do not have the **suits** array, we could use the following **if/else** block of code:

```
if (deck[i] / 13 == 0)
    System.out.print("suit is Spades");
else if (deck[i] / 13 == 1)
    System.out.print("suit is Hearts");
else if (deck[i] / 13 == 2)
    System.out.print("suit is Diamonds");
else
    System.out.print("suit is Clubs");
```

- ◆ Notice how using arrays greatly simplified our solution for this program



Copying Arrays

➡ Often, you need to duplicate (copy) an array or part of an array

➡ Assume we have one array:

```
list1
```

➡ And we want to duplicate this array into:

```
list2
```

➡ Maybe you will do the following:

```
list2 = list1;
```

➡ **This statement does not work!**

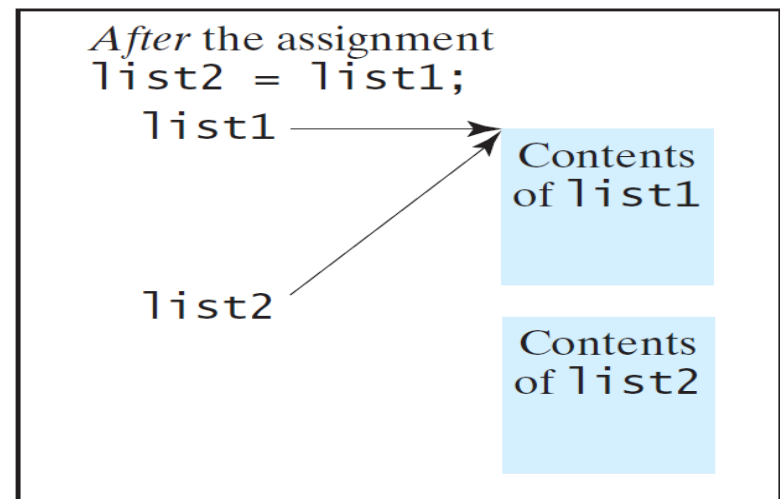
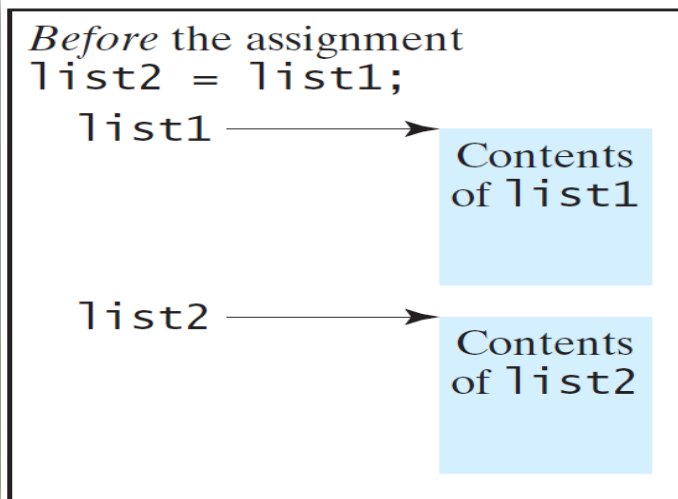


Copying Arrays (cont.)

```
list2 = list1;
```

– This copies the reference from **list1** to **list2**

◆ Now, both variables reference the same array



- After the assignment, the reference inside **list1** is saved into **list2**
 - Both arrays will point to the same memory locations
- The array previously referenced by **list2** is no longer referenced! It becomes garbage
 - This process is called *garbage collection*



Copying Arrays (cont.)

- ➡ Assignment statements can be used to copy primitive statements
- ➡ However, assignment statements **cannot be used to copy arrays**
- ➡ There are three ways to copy arrays:
 1. Use a loop to copy individual elements one by one.
 2. Use the static **arraycopy** method in the **System** class.
 3. Use the clone method to copy arrays
 - ◆ this is covered in CPCS-203



Copying Arrays (cont.)

1. Use a loop to copy individual elements one by one.
 - ◆ The following code copies the values of **sourceArray** to **targetArray** using a **for** loop:

```
int[] sourceArray = {2, 3, 1, 5, 10};  
int[] targetArray = new int[sourceArray.length];  
for (int i = 0; i < sourceArray.length; i++)  
    targetArray[i] = sourceArray[i];
```

2. Use the static **arraycopy** method in the **System** class.

– Syntax:

System.arraycopy(sourceArray, srcPos, targetArray, tarPos, length)

- **srcPos** is the starting position of **sourceArray**
- **tarPos** is the starting position of **targetArray**
- **length** is the number of elements to be copied

```
int[] sourceArray = {2, 3, 1, 5, 10};  
int[] targetArray = new int[sourceArray.length];  
System.arraycopy(sourceArray, 0, targetArray, 0, sourceArray.length);
```

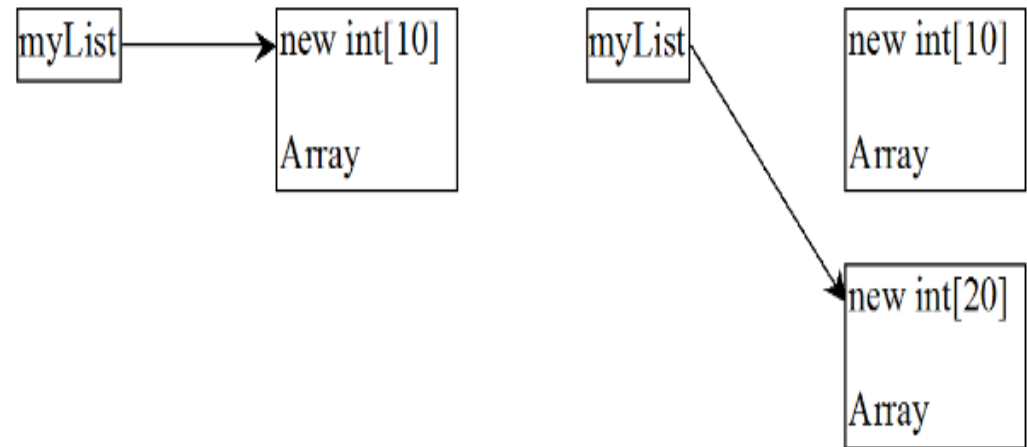


Check Point

- ➡ Once an array is created, its size cannot be changed. What does the following code do?

```
int[] myList;  
myList = new int[10];  
// You have added the 10 values...but now you want to add more  
myList = new int[20];
```

- The code will make a NEW array of size 20
- The reference for this new array will be saved into the variable **myList**.
- The old array of size 10 will be deleted by garbage collector





Passing Arrays to Methods

➡ Just like you can pass primitive values to methods, you can also pass arrays to methods

➡ Example method:

```
public static void printArray(int[] list) {  
    for (int i = 0; i < array.length; i++)  
        System.out.print(array[i] + " ");  
}
```

➡ From main we can invoke the method:

```
int[] list = {3, 1, 2, 6, 4, 2};  
printArray(list);
```

```
public static void printArray(int[] list) {  
    for (int i = 0; i < array.length; i++)  
        System.out.print(array[i] + " ");  
}
```



Passing Arrays to Methods (cont.)

- When passing an array to a method, the reference of the array is passed to the method
- Important differences between passing primitive data types and passing arrays:
 - ◆ For an argument of a primitive type, the actual value is passed *pass-by-value*
 - ◆ For an argument of an array type, the reference to an array is passed *pass-by-reference*



Passing Arrays to Methods (cont.)

- So when you pass an array to a method, you are not passing the actual values
 - ◆ You are passing a reference (an address) to the memory of the array
- What is the result of this?
 - ◆ Inside the method, the array accessed inside the method is the actual/original array in memory
 - ◆ Therefore, if you change the array inside the method, you will also see the changes outside the method



Passing Arrays to Methods (cont.)

➡ Example of *pass-by-value* & *pass-by-reference* :

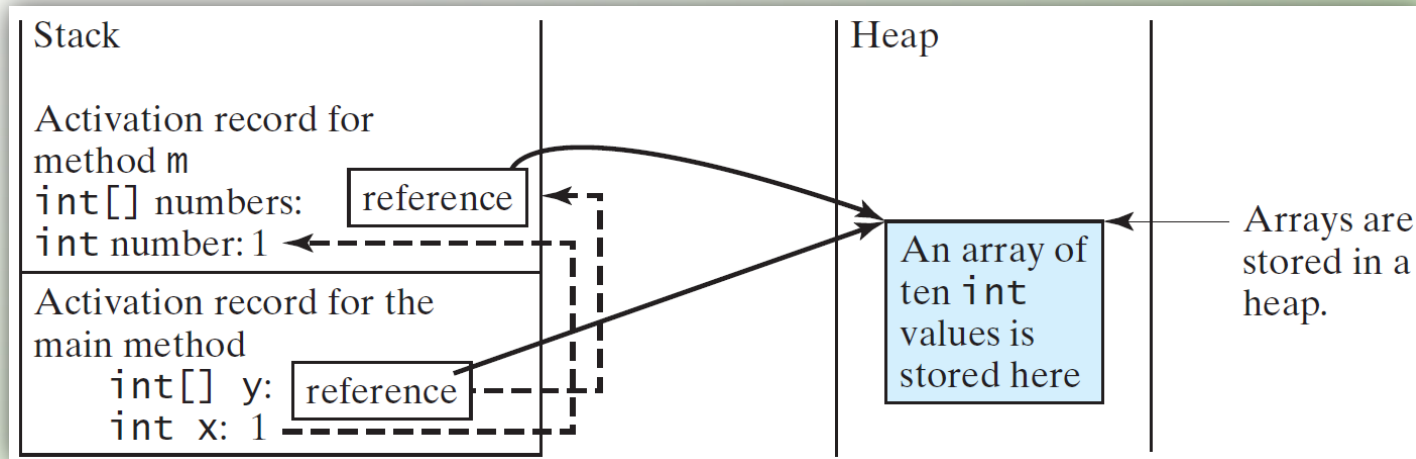
```
public class Test {  
    public static void main(String[] args) {  
        int x = 1; // x represents an int value  
        int[] y = new int[10]; // y represents an array of int values  
  
        m(x, y); // Invoke m with arguments x and y  
  
        System.out.println("x is " + x);  
        System.out.println("y[0] is " + y[0]);  
    }  
  
    public static void m(int number, int[] numbers) {  
        number = 1001; // Assign a new value to number  
        numbers[0] = 5555; // Assign a new value to numbers[0]  
    }  
}
```



```
x is 1  
y[0] is 5555
```

Passing Arrays to Methods (cont.)

➡ Method Call Stack:



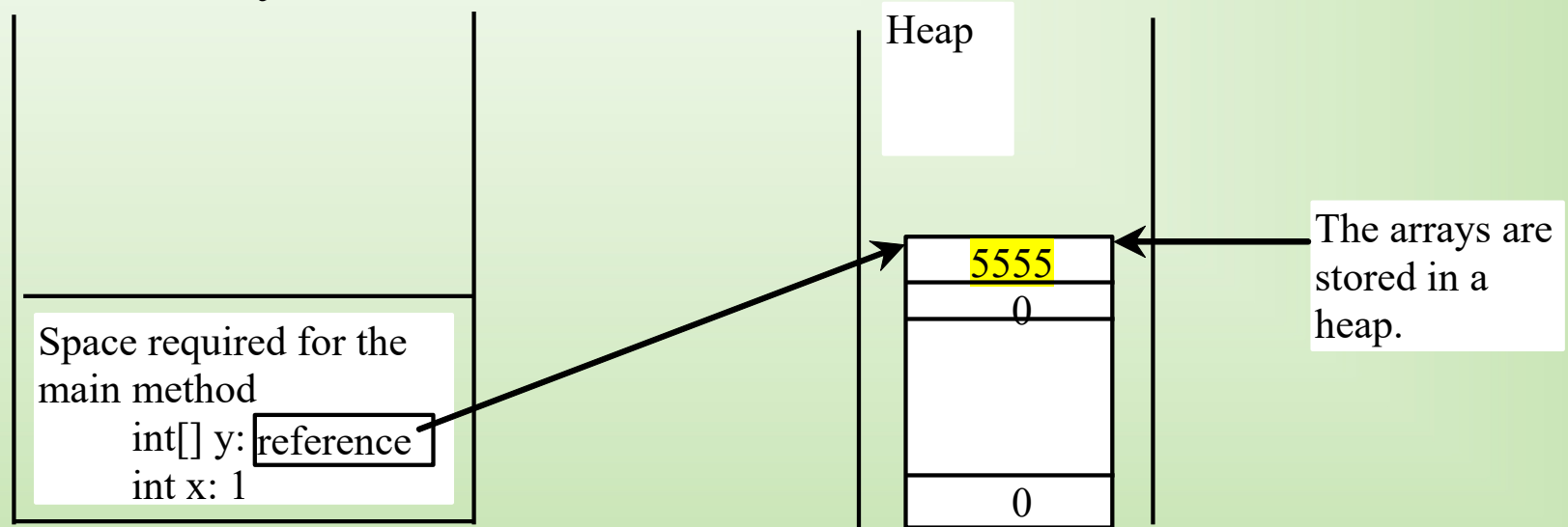
- When invoking **m(x, y)**, the values of **x** and **y** are passed to **number** and **numbers**.
- Since **y** contains the reference value to the array, **numbers** now contains the same reference value to the same array.



Passing Arrays to Methods (cont.)

➡ Heap memory:

- Arrays are stored in memory in a special location
- This location is called the heap
- The heap is used to store all dynamically allocated memory





Passing Arrays to Methods (cont.)

➡ Another example:

- The following program again illustrates the difference between passing a primitive data type value and an array reference variable
- The program contains two swap methods:
 - ◆ The first method attempts to swap two int arguments
 - This method fails...no swap will occur within main
 - ◆ The second method attempts to swap the first two elements in an array argument
 - This method works as expected.
 - How?
 - Because the swap occurs in heap memory and is permanent!



```
1 public class TestPassArray {
2     /** Main method */
3     public static void main(String[] args)
4     { int[] a = {1, 2};
5         // Swap elements using the swap method
6         System.out.println("Before invoking swap");
7         System.out.println("array is {" + a[0] + ", " + a[1] + "}");
8         swap(a[0], a[1]);
9         System.out.println("After invoking swap");
10        System.out.println("array is {" + a[0] + ", " + a[1] + "}");
11        // Swap elements using the swapFirstTwoInArray method
12        System.out.println("Before invoking swapFirstTwoInArray");
13        System.out.println("array is {" + a[0] + ", " + a[1] + "}");
14        swapFirstTwoInArray(a);
15        System.out.println("After invoking swapFirstTwoInArray");
16        System.out.println("array is {" + a[0] + ", " + a[1] + "}");
17    }
18    /** Swap two variables */
19    public static void swap(int n1, int n2)
20    { int temp = n1;
21        n1 = n2;
22        n2 = temp;
23    }
24    /** Swap the first two elements in the array */
25    public static void swapFirstTwoInArray(int[] array)
26    { int temp = array[0];
27        array[0] = array[1];
28        array[1] = temp;
29    } }
```

Passing Arrays to Methods (cont.)

➡ Run the Program:

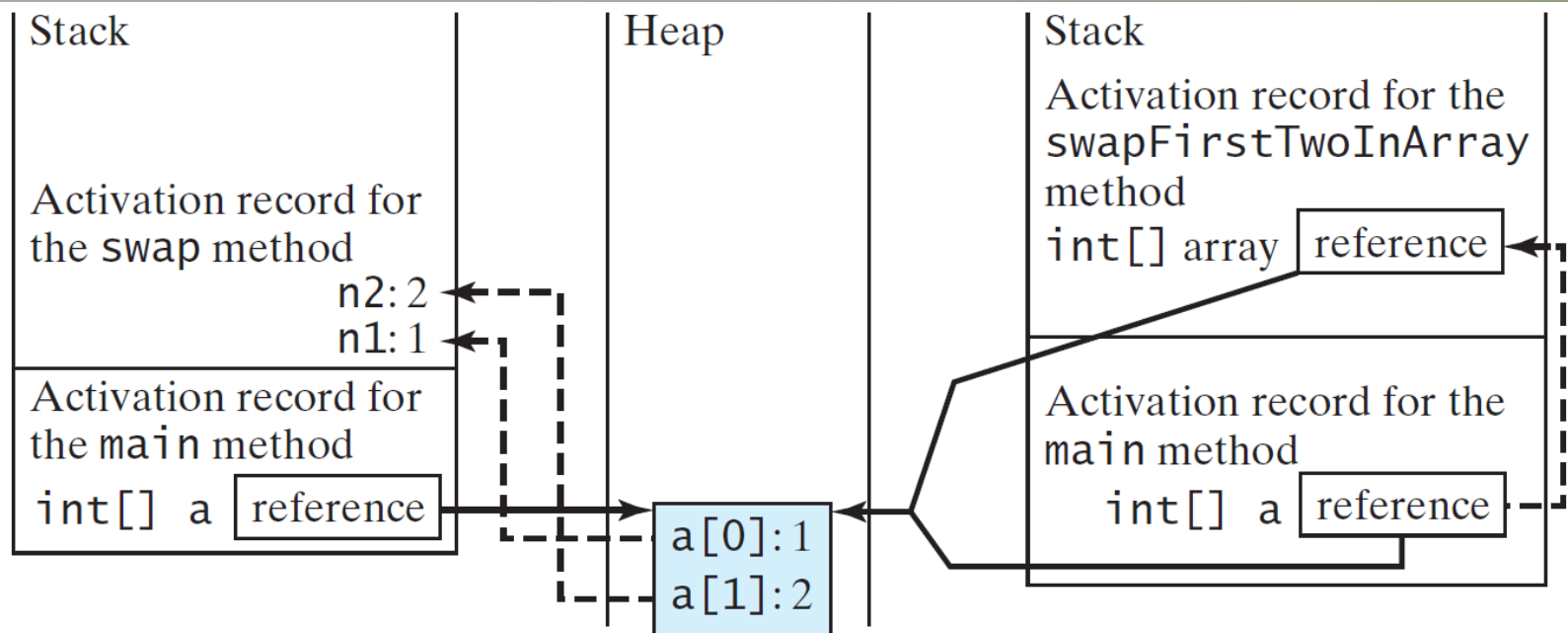
```
Before invoking swap  
array is {1, 2}  
After invoking swap  
array is {1, 2}  
Before invoking swapFirstTwoInArray  
array is {1, 2}  
After invoking swapFirstTwoInArray  
array is {2, 1}
```



[Click here](#) to view and trace code

Passing Arrays to Methods

– Details:



Invoke `swap(int n1, int n2)`. The primitive type values in `a[0]` and `a[1]` are passed to the `swap` method.

The arrays are stored in a heap.

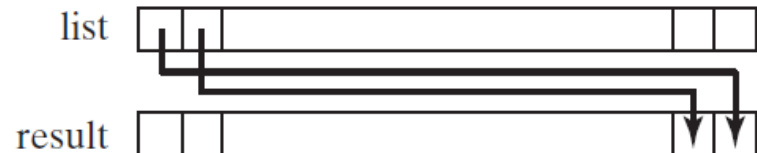
Invoke `swapFirstTwoInArray(int[] array)`. The reference value in `a` is passed to the `swapFirstTwoInArray` method.



Returning an Array from a Method

- ➡ A method can also return an array
 - When a method returns an array, the reference of the array is returned
- ➡ Consider the following code:

```
1 public static int[] reverse(int[] list) {  
2     int[] result = new int[list.length];  
3  
4     for (int i = 0, j = result.length - 1;  
5         i < list.length; i++, j--) {  
6         result[j] = list[i];  
7     }  
8  
9     return result;  
10 }
```





Returning an Array from a Method

☞ Details:

- Line 2 creates a new array **result**
- Lines 4 – 7 copy elements from array **list** to array **result**
- Line 9 returns the reference of the new array **result**

☞ Inside main:

```
int[ ] list1 = {1, 2, 3, 4, 5, 6};  
int[ ] list2 = reverse(list1);
```

Trace the reverse Method

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

Declare result and create array

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1;  
        i < list.length; i++, j--) {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

list

1	2	3	4	5	6
---	---	---	---	---	---

result

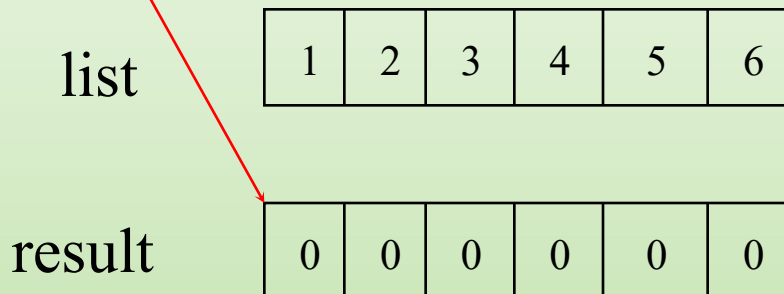
0	0	0	0	0	0
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1;  
         i < list.length; i++, j--) {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i = 0 and j = 5

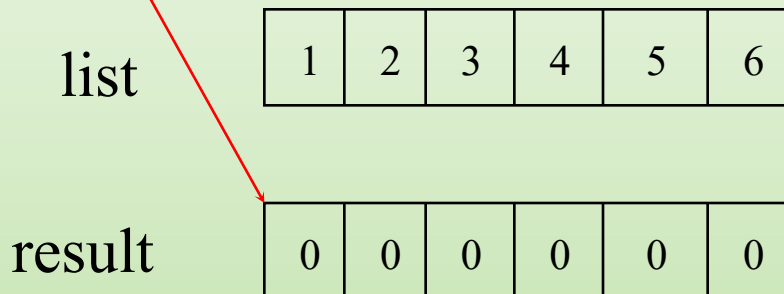


Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i (= 0) is less than 6

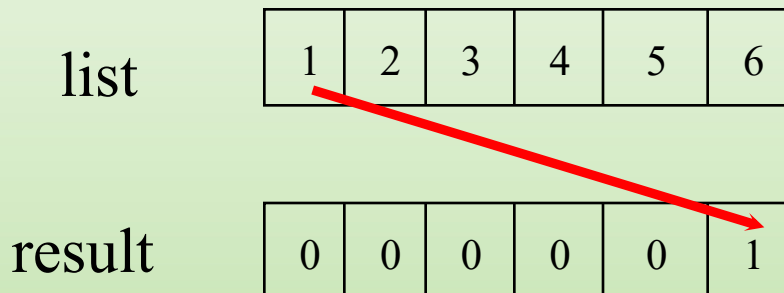


Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i = 0 and j = 5
Assign list[0] to result[5]



Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

After this, i becomes 1 and j becomes 4

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

list

1	2	3	4	5	6
---	---	---	---	---	---

result

0	0	0	0	0	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i (=1) is less than 6

list

1	2	3	4	5	6
---	---	---	---	---	---

result

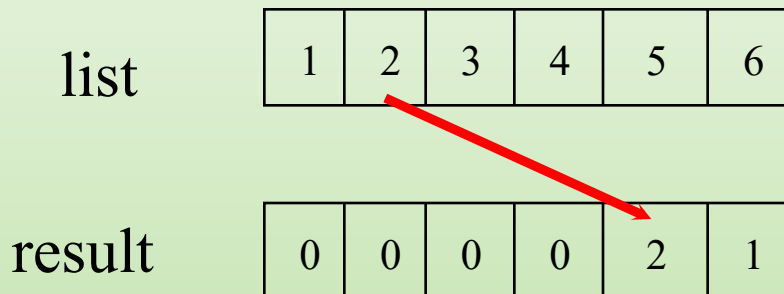
0	0	0	0	0	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i = 1 and j = 4
Assign list[1] to result[4]



Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

After this, i becomes 2 and
j becomes 3

list

1	2	3	4	5	6
---	---	---	---	---	---

result

0	0	0	0	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};
```

```
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i (=2) is still less than 6

list

1	2	3	4	5	6
---	---	---	---	---	---

result

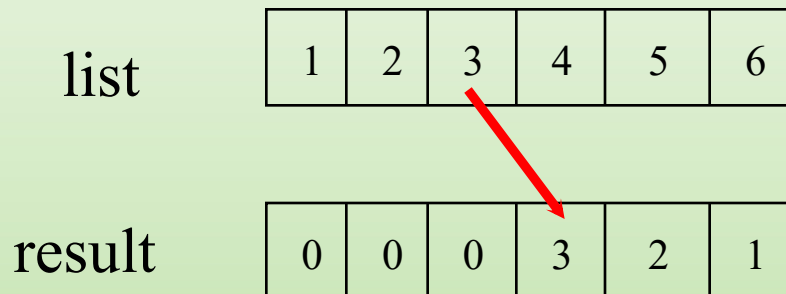
0	0	0	0	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

$i = 2$ and $j = 3$
Assign $list[i]$ to $result[j]$



Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

After this, i becomes 3 and
j becomes 2

list

1	2	3	4	5	6
---	---	---	---	---	---

result

0	0	0	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i (=3) is still less than 6

list

1	2	3	4	5	6
---	---	---	---	---	---

result

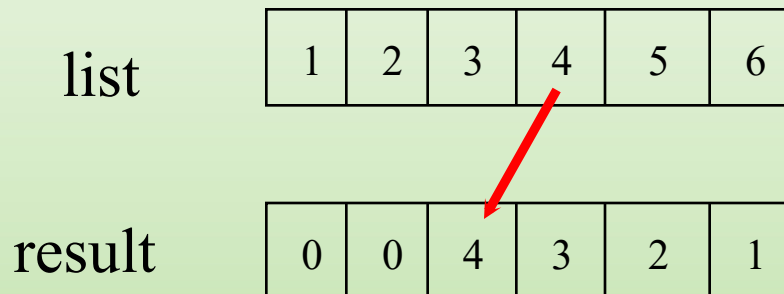
0	0	0	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i = 3 and j = 2
Assign list[i] to result[j]



Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

After this, i becomes 4 and
j becomes 1

list

1	2	3	4	5	6
---	---	---	---	---	---

result

0	0	4	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};
```

```
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];
```

```
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)
```

```
    {  
        result[j] = list[i];
```

```
    }  
  
    return result;
```

```
}
```

i (=4) is still less than 6

list

1	2	3	4	5	6
---	---	---	---	---	---

result

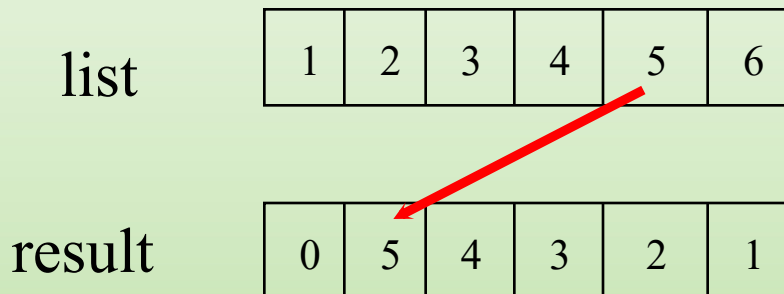
0	0	4	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i = 4 and j = 1
Assign list[i] to result[j]



Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

After this, i becomes 5 and
j becomes 0

list

1	2	3	4	5	6
---	---	---	---	---	---

result

0	5	4	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

i (=5) is still less than 6

list

1	2	3	4	5	6
---	---	---	---	---	---

result

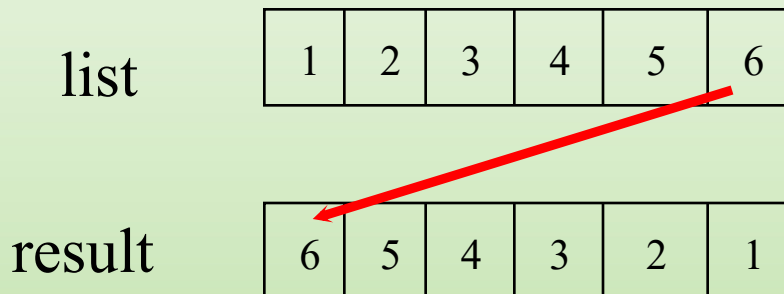
0	5	4	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

$i = 5$ and $j = 0$
Assign $list[i]$ to $result[j]$



Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

After this, i becomes 6 and
j becomes -1

list

1	2	3	4	5	6
---	---	---	---	---	---

result

6	5	4	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

$i (=6) < 6$ is false. So exit the loop.

list

1	2	3	4	5	6
---	---	---	---	---	---

result

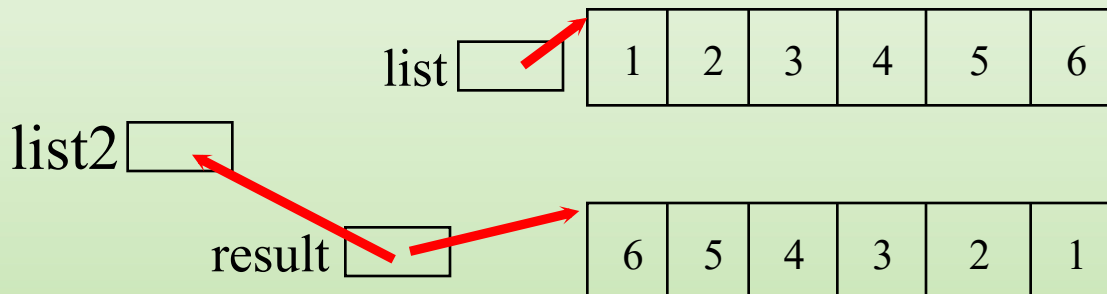
6	5	4	3	2	1
---	---	---	---	---	---

Trace the reverse Method, cont.

```
int[] list1 = {1, 2, 3, 4, 5, 6};  
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--)  
    {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

Return result





Program 3: Count Letters In Array

- ➡ Given an array of letters, count the number of times each letter appears in the array.
 - Generate an array of 100 lowercase letters (randomly)
 - Count the occurrence of each letter in the array

- ➡ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 3: Count Letters In Array

☞ Step 1: Problem-solving Phase

– Details:

- ◆ How can we generate a random lowercase letter?
- ◆ We did this in Chapter 6
 - `getRandomLowerCaseLetter()` method

```
/** Generate a random character between ch1 and ch2 */
public static char getRandomCharacter(char ch1, char ch2) {
    return (char)(ch1 + Math.random() * (ch2 - ch1 + 1));
}

/** Generate a random lowercase letter */
public static char getRandomLowerCaseLetter() {
    return getRandomCharacter('a', 'z');
}
```



Program 3: Count Letters In Array

☞ Step 1: Problem-solving Phase

– Details:

- ◆ Now that you have the random array of lowercase letters, how can we count the numbers of each letter?
 - Create an array, called **counts**, of 26 **int** values
 - Each index of this array represents one alphabetical character
 - **counts[0]** represents the number of "a"
 - **counts[1]** represents the number "b"
 - ...
 - **counts[25]** represents the number "z"



Program 3: Count Letters In Array

☞ Step 1: Problem-solving Phase

– Algorithm:

◆ In main, we will use mostly methods

- We use a method to create the array of random letters
 - This method **returns** a reference to the new array
- We use a method to then print the array of random letters
 - This method is a **void** method
 - It simply prints 20 letters per line
- We use a method to then count the occurrences of each letter
 - This method **returns** a reference to the **counts** array
- Finally, we use a method to display the results of **counts** array
 - This method is a **void** method



➡ Step 2: Implementation of Methods

```
1  public class CountLettersInArray {
2      /** Main method */
3      public static void main(String[] args) {
4          // Declare and create an array
5          char[] chars = createArray();
6
7          // Display the array
8          System.out.println("The lowercase letters are:");
9          displayArray(chars);
10
11         // Count the occurrences of each letter
12         int[] counts = countLetters(chars);
13
14         // Display counts
15         System.out.println();
16         System.out.println("The occurrences of each letter are:");
17         displayCounts(counts);
18     }
19 }
```



```
20  /** Create an array of characters */
21  public static char[] createArray() {
22      // Declare an array of characters and create it
23      char[] chars = new char[100];
24
25      // Create lowercase letters randomly and assign
26      // them to the array
27      for (int i = 0; i < chars.length; i++)
28          chars[i] = RandomCharacter.getRandomLowerCaseLetter();
29
30      // Return the array
31      return chars;
32  }
33
34  /** Display the array of characters */
35  public static void displayArray(char[] chars) {
36      // Display the characters in the array 20 on each line
37      for (int i = 0; i < chars.length; i++) {
38          if ((i + 1) % 20 == 0)
39              System.out.println(chars[i]);
40          else
41              System.out.print(chars[i] + " ");
42      }
43  }
44
```



```
45  /** Count the occurrences of each letter */
46  public static int[] countLetters(char[] chars) {
47      // Declare and create an array of 26 int
48      int[] counts = new int[26];
49
50      // For each lowercase letter in the random array, count it
51      for (int i = 0; i < chars.length; i++)
52          counts[chars[i] - 'a']++;
53
54      return counts;
55  }
56
57  /** Display counts */
58  public static void displayCounts(int[] counts) {
59      for (int i = 0; i < counts.length; i++) {
60          if ((i + 1) % 10 == 0)
61              System.out.println(counts[i] + " " + (char)(i + 'a'));
62          else
63              System.out.print(counts[i] + " " + (char)(i + 'a') + " ");
64      }
65  }
66 }
```

- ◆ What does **counts[chars[i] - 'a']++** do?
 - This would then be **counts['a' - 'a']++**
 - Which equates to **counts[0]++**
 - This would increment index 0 of counts array, which is correct!!!

Program 3: CountLettersInArray

➡ Run the Program:

The lowercase letters are:

```
e y l s r i b k j v j h a b z n w b t v  
s c c k r d w a m p w v u n q a m p l o  
a z g d e g f i n d x m z o u l o z j v  
h w i w n t g x w c d o t x h y v z y z  
q e a m f w p g u q t r e n n w f c r f
```

The occurrences of each letter are:

```
5 a 3 b 4 c 4 d 4 e 4 f 4 g 3 h 3 i 3 j  
2 k 3 l 4 m 6 n 4 o 3 p 3 q 4 r 2 s 4 t  
3 u 5 v 8 w 3 x 3 y 6 z
```



[Click here](#) to view and trace code



Check Point

☞ True or false.

When an array is passed to a method, a new array is created inside the method?

- Of course, the answer is false.
- There is only one array
 - ◆ It is stored on the Heap!
- When you pass an array to a method, you are simply passing a reference to the stored array on the heap.



Check Point

➡ Show the output of the following two programs:

```
public class Test {  
    public static void main(String[] args) {  
        int number = 0;  
        int[] numbers = new int[1];  
  
        m(number, numbers);  
  
        System.out.println("number is " + number  
            + " and numbers[0] is " + numbers[0]);  
    }  
  
    public static void m(int x, int[] y) {  
        x = 3;  
        y[0] = 3;  
    }  
}
```

(a)

```
public class Test {  
    public static void main(String[] args) {  
        int[] list = {1, 2, 3, 4, 5};  
        reverse(list);  
        for (int i = 0; i < list.length; i++)  
            System.out.print(list[i] + " ");  
    }  
  
    public static void reverse(int[] list) {  
        int[] newList = new int[list.length];  
  
        for (int i = 0; i < list.length; i++)  
            newList[i] = list[list.length - 1 - i];  
  
        list = newList;  
    }  
}
```

(b)



Searching Arrays

- ☞ Searching an array is a very common task
 - You want to find a specific value in an array
 - ◆ Perhaps looking for a specific student name or ID
 - How to you do this?
 - ◆ Basic approach: use a for loop
 - ◆ For each iteration of the loop, check if that cell of the array contains the value you are looking for
 - If so, return true (or the index where it was found)
 - Otherwise, return false
 - This is the basic idea of Linear Search



Linear Search

- ☞ We use a **for** loop to test every element inside the array
 - If we find a match, the linear search method will return the index of the element where the match was found
 - If no match is found by the end of the **for** loop, the linear search method returns a **-1**
 - ◆ Why?
 - ◆ Because there is no index **-1**
 - ◆ Therefore, inside main, if the method returns a **-1**, you know that the value was not found



Linear Search

👉 Code:

LISTING 7.6 LinearSearch.java

```
1 public class LinearSearch {
2     /** The method for finding a key in the list */
3     public static int linearSearch(int[] list, int key) {
4         for (int i = 0; i < list.length; i++) {
5             if (key == list[i])
6                 return i;
7         }
8         return -1;
9     }
10 }
```

[0] [1] [2] ...
list

--	--	--	--	--	--

key Compare key with list[i] for i = 0, 1, ...

– Trace with these statements:

```
int[] list = {1, 4, 4, 2, 5, -3, 6, 2};
int i = linearSearch(list, 4); // Returns 1
int j = linearSearch(list, -4); // Returns -1
int k = linearSearch(list, -3); // Returns 5
```

Linear Search Animation

Key

List

3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8



Linear Search

☞ Is linear search efficient (fast)?

– No!

◆ Linear search is slow!

– Why?

- ◆ Because, worst scenario, linear search must test each and every element...perhaps the match is the last index
- ◆ Or even worse than this, perhaps there is no match!
- ◆ So if there are **n** elements in the array, worst scenario, linear search must test/compare each element
- ◆ This is slow!

☞ We can do much better by using **Binary Search**!



Binary Search

➡ Binary Search is a famous searching algorithm

- It is VERY fast!
- There is one important condition:
 - ◆ The array must be sorted!

– Examine this array:

2 4 7 10 11 45 50 59 60 66 69 70 79

- We have 13 elements
- The array is sorted
- And we are searching a key/value in the array



Binary Search

➡ Algorithm:

2	4	7	10	11	45	50	59	60	66	69	70	79
0	1	2	3	4	5	6	7	8	9	10	11	13

low

Find the middle element, which is at index 6.

mid

high

- ◆ If the key matches that middle element, return the index to indicate that you found the item, How do you find the middle index?

$$\text{mid} = (\text{low} + \text{high}) / 2;$$

- ◆ If the key is less than the middle element, this means it MUST be on the left side
 - from index 0 to index 5
- ◆ If the key is greater than the middle element, this means it MUST be on the right side
 - from index 7 to index 13



Binary Search

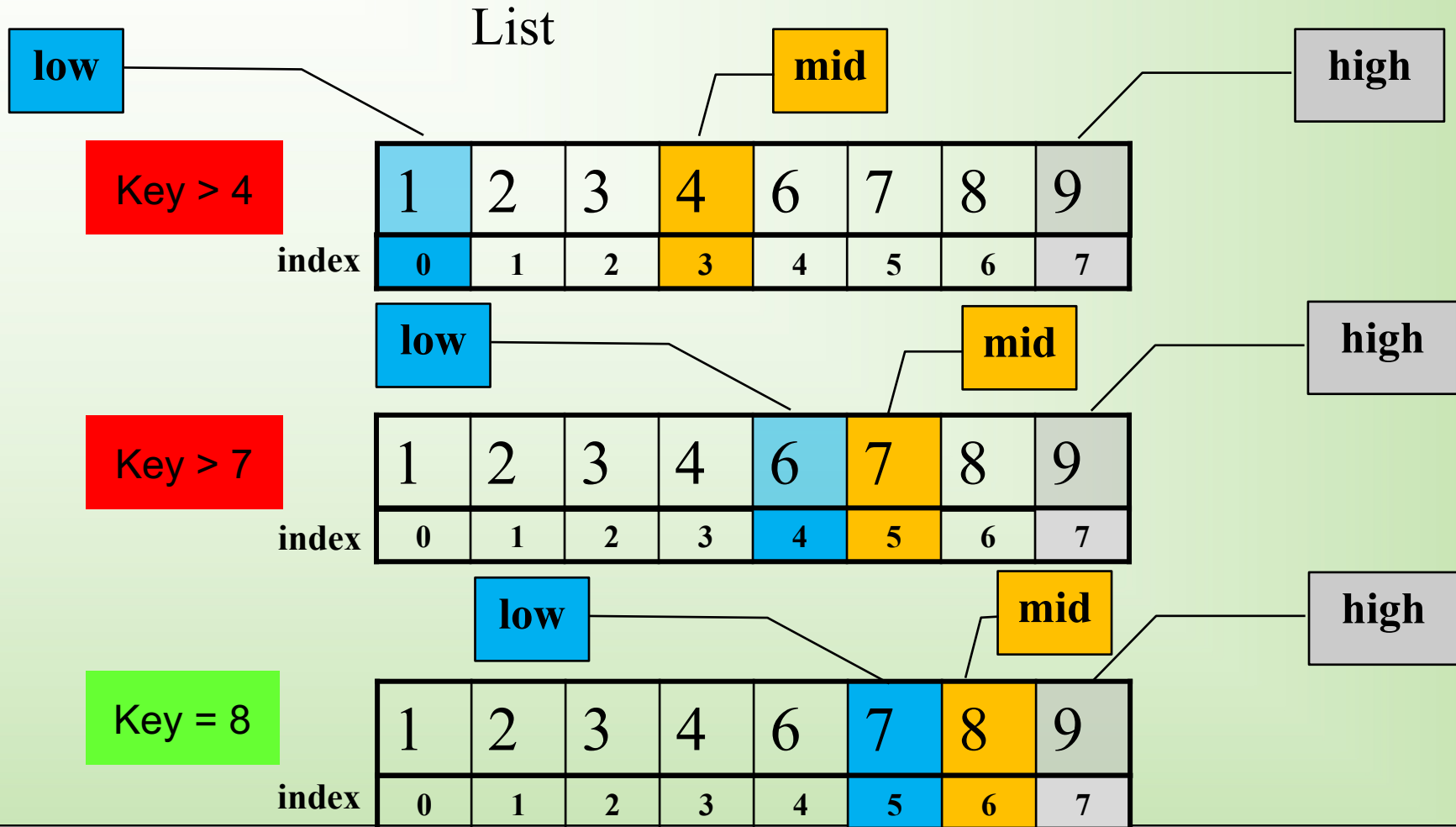
☞ Code:

```
public static int binarySearch(int[] list, int key) {
    int low = 0;
    int high = list.length-1;

    while (low <= high) {
        int mid = (low + high)/2;
        if (key == list[mid])
            return mid; // the index of key
        else if (key < list[mid])
            high = mid - 1;
        else // if (key > list[mid]...change low
            low = mid + 1;
    }
    return -1; // this only happens if not found
}
```

Binary Search

If we are looking for **key = 8**





Sorting Arrays

- ☞ Sorting is also a very common task in computer programming
 - Think of all the applications you have that use sorting!
 - ◆ Your phone book!
- ☞ There are many different sorting algorithms
- ☞ In this course, we introduce a very basic one:
 - Selection Sort



Sorting Arrays

☞ Selection Sort:

- The algorithm to sort n numbers is as follows:
 - ◆ There is a FOR loop that iterates from $i = 0$ to $i = n-1$
 - ◆ FOR the i^{th} element (as i ranges from 0 to $n-1$)
 - 1) Determine the smallest element in the rest of the array
 - To the right of the i^{th} element
 - 2) Swap the current i^{th} element with the element identified in part (1) above (the smallest element)
 - ◆ Essentially:
 - The algorithm first picks the smallest element and swaps it into the first location.
 - Then it picks the next smallest element and swaps it into the next location, etc.



Sorting Arrays

☞ Selection Sort:

– Example:

◆ Here is an array of 5 integers

20	8	5	10	7
0	1	2	3	4

$i = 0$

◆ 5 (at index 2) is the smallest element

– from the range $i = 0$ to 4

◆ So SWAP the value at index 2 with the value at index 0

– SWAP the 5 and the 20

5	8	20	10	7
0	1	2	3	4



Sorting Arrays

☞ Selection Sort:

– Example:

◆ Here is an array of 5 integers

5	8	20	10	7
0	1	2	3	4

$i = 1$

◆ 7 (at index 4) is the smallest element

– from the range $i = 1$ to 4

◆ So SWAP the value at index 4 with the value at index 1

– SWAP the 7 and the 8

5	7	20	10	8
0	1	2	3	4



Sorting Arrays

☞ Selection Sort:

– Example:

◆ Here is an array of 5 integers

5	7	20	10	8
0	1	2	3	4

$i = 2$

◆ 8 (at index 4) is the smallest element

– from the range $i = 2$ to 4

◆ So SWAP the value at index 4 with the value at index 2

– SWAP the 8 and the 20

5	7	8	10	20
0	1	2	3	4



Sorting Arrays

☞ Selection Sort:

– Example:

◆ Here is an array of 5 integers

5	7	8	10	20
0	1	2	3	4

$i = 3$

◆ 10 (at index 3) is the smallest element

– from the range $i = 3$ to 4

◆ So SWAP the value at index 3 with the value at index 3

– SWAP the 10 and the 10 (so no swap really happened here)

5	7	8	10	20
0	1	2	3	4



Sorting Arrays

☞ Selection Sort:

– Example:

◆ Here is an array of 5 integers

5	7	8	10	20
0	1	2	3	4

$i = 4$

◆ 20 (at index 4) is the smallest element

– from the range $i = 4$ to 4

◆ So SWAP the value at index 4 with the value at index 4

– SWAP the 20 and the 20 (so no swap really happened here)

5	7	8	10	20
0	1	2	3	4



Sorting Arrays

☞ Selection Sort:

– Example:

◆ Here is an array of 5 integers

5	7	8	10	20
0	1	2	3	4

- ◆ The array is now in sorted order
- ◆ We see that the last iteration was not even necessary
 - In code our for loop could look like this:
`for (i = 0; i < n-1; i++)`
 - So it won't even iterate on the n-1 step



Selection Sort Code

LISTING 7.8 SelectionSort.java

```
1  public class SelectionSort {
2      /** The method for sorting the numbers */
3      public static void selectionSort(double[] list) {
4          for (int i = 0; i < list.length - 1; i++) {
5              // Find the minimum in the list[i..list.length-1]
6              double currentMin = list[i];
7              int currentMinIndex = i;
8
9              for (int j = i + 1; j < list.length; j++) {
10                 if (currentMin > list[j]) {
11                     currentMin = list[j];
12                     currentMinIndex = j;
13                 }
14             }
15
16             // Swap list[i] with list[currentMinIndex] if necessary
17             if (currentMinIndex != i) {
18                 list[currentMinIndex] = list[i];
19                 list[i] = currentMin;
20             }
21         }
22     }
23 }
```



Arrays Class

- ☞ Clearly there are many useful things to do with arrays
 - Searching, sorting, and more!
- ☞ Therefore, Java provides an Arrays class with many useful methods

java.util.Arrays

- ☞ This class allows you:
 - to sort,
 - to perform binary search,
 - to test if two arrays are equal, and more!



Arrays Class

☞ Sorting:

- Consider the following code:

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};  
java.util.Arrays.sort(numbers);  
java.util.Arrays.parallelSort(numbers);
```

- As you see, there are two sorting methods:

`Arrays.sort()`

`Arrays.parallelSort()`

- The difference is that `parallelSort()` is more efficient if your computer has multiple processors



Arrays Class

☞ Binary Search:

- Condition: array must be sorted in increasing order
 - ◆ If so, we can use the built-in `binarySearch()` method

```
int[] list = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};

System.out.println("1. Index is " +
    java.util.Arrays.binarySearch(list, 11));
System.out.println("2. Index is " +
    java.util.Arrays.binarySearch(list, 12));

char[] chars = {'a', 'c', 'g', 'x', 'y', 'z'};
System.out.println("3. Index is " +
    java.util.Arrays.binarySearch(chars, 'a'));
System.out.println("4. Index is " +
    java.util.Arrays.binarySearch(chars, 't'));
```



Arrays Class

☞ Content Equality:

- Java also gives us a method to test if two arrays have the same content

```
int[] list1 = {2, 4, 7, 10};  
int[] list2 = {2, 4, 7, 10};  
int[] list3 = {4, 2, 7, 10};  
System.out.println(java.util.Arrays.equals(list1, list2)); // true  
System.out.println(java.util.Arrays.equals(list2, list3)); // false
```

Chapter 7:

Arrays

(Single Dimensional)

